UPX-ADR018 - Messaging tool

Date: 2025-8-26

Title

Decision on the messaging tool for UPX application

Status

New

Context

As part of the UPX solution, a messaging tool is needed to share events and messages occurring in the UPX application both within and outside the ecosystem.

Requirement

There is a capability within Portal2 where the case/request are saved partially i.e. case is still not submitted. External user can resume the case anytime later from where is last saved the case/request. There is a business requirement to show the case which is partially saved on UPX dashboard

There are capabilities within UPX where a messaging tool will be used to share message / events occurring in the application.

Following are the areas where the tools will be utilized

- Submit the authorization request
- Get latest updates for authorization from downstream application (Document Upload Status, UCX, UPADS, Letters)
- Get the provider details from ERSS to calculate the provider frecency score

Architecture Characteristic to Consider

- Realtime solution
- Data Consistency
- Feasibility
- Observability
- Time to market

Solution Options

Here's a detailed comparison of event-driven services to help you quickly evaluate which service meets your security, reliability, and compliance needs for healthcare and enterprise applications.

Primary factor's affecting our decision

Secondary factory affecting our decision

Feature / Service	Confluent Kafka: eviCore managed (current state)	Event Grid	Service Bus	Event Hubs	Storage Queues
Primary Use Case	High-throughput streaming, analytics, event sourcing	Event notification, serverless triggers, resource events	Reliable messaging, pub/sub, workflow integration	Big data streaming, telemetry, analytics	Simple decoupling, basic queueing
Protocol(s)	Kafka TCP (custom binary), REST Proxy, Confluent REST, Connect, Schema Registry	HTTPS (REST, WebHooks), AMQP (preview)	AMQP, HTTPS (REST), SBMP	AMQP, HTTPS (REST), Kafka (protocol head)	HTTPS (REST), SMB (legacy)
Event Delivery Model	Pull (consumer groups read from topics)	Push (event to endpoint)	Pull (client receives from queue/topic)	Pull (client reads from partition)	Pull (client reads from queue)
Ordering Guarantee	Yes (per partition, not global)	No (events may arrive out of order)	Yes (FIFO in queue, FIFO in topic with sessions)	Yes (per partition, not global)	No (best effort, not strict FIFO)

At-Least-Once Delivery	Yes (default; exactly-once with config)	Yes (with retries)	Yes	Yes	Yes
Exactly-Once Delivery	Yes (with idempotent producer and transactional consumer)	No	Yes (with deduplication, Premium tier)	No	No
Dead-Lettering	No native, but can implement with dead-letter topics	No (but can route failed events elsewhere)	Yes (dead-letter queues for both queues and topics)	No (consumer must handle failures)	No
Message Size Limit	Configurable (default 1 MB, can be increased)	1 MB	256 KB (Standard), 1 MB (Premium)	1 MB	64 KB
Throughput	Millions of events/sec (scalable with cluster size)	Millions of events/sec (scalable)	High (Premium tier for very high throughput)	Millions of events/sec (partitioned)	Moderate
Latency	Low (ms to seconds, depends on config)	Low (near real-time)	Low to moderate	Low	Moderate
Durability	Yes (configurable retention, disk-based)	No (event notification only, not persisted)	Yes (messages persisted until consumed or expired)	Yes (events retained for 1-7 days, configurable)	Yes (messages persisted until deleted)

Pub/Sub Support	Yes (topics, partitions, consumer groups)	Yes (event subscriptions)	Yes (topics/subscriptions)	No native pub/sub, but multiple consumers per partition	No
Filtering	No native, but can implement at consumer or with Kafka Streams	Yes (event type, subject, advanced filters)	No (all messages delivered to queue/topic)	No (all events delivered to partition)	No
Batching	Yes (producer and consumer batching)	No	Yes (receive/send in batches)	Yes (send/receive in batches)	No
Retry Policy	No native, must be handled by consumer logic	Yes (configurable, exponential backoff)	Yes (configurable, dead-letter on failure)	No native retry, consumer must handle	No native retry, client must handle
Integration	Wide ecosystem: Kafka Connect, Streams, REST Proxy, Schema Registry, ksqlDB	Native Azure services, custom webhooks	Azure, on-prem, hybrid, custom apps	Azure Stream Analytics, Databricks, custom consumers	Azure Functions, custom apps

Security	TLS, SASL, Kerberos, ACLs, RBAC (Confluent), audit logs, network/firewall	Managed identities, SAS, webhook validation	RBAC, SAS, managed identities, firewall, encryption	RBAC, SAS, managed identities, encryption	SAS, storage account keys, managed identities
RBAC Support	Yes (Confluent RBAC, fine-grained role and resource-based access)	Yes (Azure RBAC for topics, event subscriptions, and access policies)	Yes (Azure RBAC for namespaces, queues, topics, subscriptions)	Yes (Azure RBAC for namespaces, event hubs, consumer groups)	Yes (Azure RBAC for storage account and queue access)
Geo-Replication	Yes (Confluent Replicator, multi-datacenter, cluster linking) - Replication in place but used as DR strategy Manually change broker details in application and manually change OffSet	No	Yes (Premium tier, geo-disaster recovery)	Yes (Geo-disaster recovery, Dedicated tier)	No
Infrastructure	Shared across organization (eviCore, evernorth, Cigna)	Independent	Independent	Independent	Independent

Blast radius	Across organization with eviCore	Independent	Independent	Independent	Independent
SLA's	not defined by eviCore kafka team N/A (self-managed; SLA depends on eviCore infra/ops; Confluent Platform offers 99.95%+ for enterprise support)	99.99% (system/custom topics, supported regions)	99.99% (Premium), 99.9% (Standard)	99.99% (Standard/Dedicated), 99.9% (Basic)	99.9% (General Purpose v2/v1 accounts)
Availability Zone Support	not supported on cloud VM instance. Current version	Yes (for system topics and custom topics in supported regions)	Yes (Premium tier only)	Yes (Standard and Dedicated tiers)	No
Management/ Monitoring	Confluent Control Center, JMX, Prometheus, Grafana, audit logs	Azure Monitor, built-in metrics, dead-letter events	Azure Monitor, Service Bus Explorer, dead-letter queue monitoring	Azure Monitor, built-in metrics	Azure Monitor, Storage Explorer
Pricing Model	Self-hosted: infra/ops cost; Confluent: per core/hour, subscription, or license	Per operation (first 100K free/month)	Per operation, tiered (Standard, Premium)	Per throughput unit, data retention	Per storage and transaction

Price	Shared: 1. Enterprise License 2. Azure infrastructure cost	Standard Throughput unit per hour per unit: $0.04 Price poer million MQTT operations: $1.00 Free usage per month - MQTT operations: 1,000,000 operations Price per million event operations: $0.60 Free usage per monnth - event operation: 1,000,000 operations Basic Tier Price per million operation: $0.60 Free usage per month: 100,000 operations	Messaging operations Basic Operations: $0.05 per millin operations Standard Base charge: $0.0135/ hour First 13M ops/month: included Next 87M ops (13-100M opx)/ month: $0.80 per million operations Next 2400M ops (100-2500M opx)/ month: $0.50 per million operations Over 2500M Ops/month: $0.20 per million operation Premium Hourly : $0.928/hour Geo replication: Zone1	Standard Capacity: $0.03/hour per throughput unit Ingress event: $0.028 per million events - standard Premium Capacity: $1.027/hour per throughput unit Ingress event: included Extended Retention: $0.10/GB/Month (10 TB included per CU) Dedicated Capacity: $6.849/hour per throughput unit Ingress event: included Extended Retention: $0.10/GB/Month (10 TB	

			(North America and Europe): $0.019/GB Zone2 (Asia, Oceania, Middle East, and Africa) : $0.12/GB Zone3 (South America): $0.23/GB Brokered connections Standard First 1K/month: Included Next 99K (1K-100K)/month: $0.03 per connection/month Next 400K (100K-500K)/month: $0.03 per connection/month Over 500K/month: $0.02 per connection/month	included per CU)	

			Premium Brokered connections are not charged in the premium tier.		
Max retention period	Unlimited (set retention.ms=-1 for infinite retention; limited only by disk space)			90 Days premium, Dedicated, 7 days standard	
HIPAA Compliance	Possible (Confluent supports security/audit, but you must validate & document)	Yes (with proper configuration, BAA required)	Yes (with proper configuration, BAA required)	Yes (with proper configuration, BAA required)	Yes (with proper configuration, BAA required)
Typical Healthcare Use	HL7/FHIR event streaming, device telemetry, integration bus, analytics	Resource change notification, audit, workflow triggers	HL7/FHIR messaging, EHR integration, reliable clinical workflows	Device telemetry, real-time analytics, population health	Simple decoupling, background job triggers
Learning Curve	The team already knows how to use this tool.	Has learning curve	Has learning curve	Has learning curve	Has learning curve
Maintenance	The Kafka team maintains the cluster.	The team must maintain the service instance while navigating the	The team must maintain the service instance while navigating the	The team must maintain the service instance while navigating the	The team must maintain the service instance while navigating the

		learning curve.	learning curve.	learning curve.	learning curve.
Implementation complexity	The team can use the existing infrastructure for all messaging capabilities without complicating message production and consumption.	• Messaging within UPX will use this service. • Messaging from outside UPX will rely on Kafka, as other platforms and ecosystems use Kafka as their message tool.	• Messaging within UPX will use this service. • Messaging from outside UPX will rely on Kafka, as other platforms and ecosystems use Kafka as their message tool.	• Messaging within UPX will use this service. • Messaging from outside UPX will rely on Kafka, as other platforms and ecosystems use Kafka as their message tool.	• Messaging within UPX will use this service. • Messaging from outside UPX will rely on Kafka, as other platforms and ecosystems use Kafka as their message tool.

- Click here to view SLA's per application (Goal) and incident report for year 2025.
- **SLA for CW application:** 99.50% up time
- **SLA for Kafka:** There is no defined percentage, but based on availability data from the Kafka team, we can assume availability is 99.50%. Below are the tier 1 operation efficiency levels:
  - Downtime < 20 minutes: M3 : (number of incidents happened last year: @Ketan Shah getting this info)
  - Downtime between 20 and 60 minutes: M2 (number of incidents happened last year: @Ketan Shah getting this info)
  - Downtime above 60: M1 (number of incidents happened last year: @Ketan Shah getting this info)
- **Expected SLA for UPX:** Yet to come up with a number

Recommendation

- We recommend using either Confluent Cloud or Azure Event Hub for their high availability and throughput.

Decision

- The team chose Kafka as the messaging tool for UPX.
- Fallback design for tool unavailability: When Kafka is unavailable, the system will push messages to a different store and reprocess them once Kafka is available.
- UPX will produce fewer messages for its own use; most messages will support a larger ecosystem that utilizes Kafka and other messaging tools.
- Even if UPX selects an Azure service for messaging, it still depends on UPADS, which uses Kafka.
- If UPADS fails, users will be unable to proceed with build requests in UPX, ultimately disrupting the build process.

Consequences

- There is no defined SLA or replication for Kafka as a messaging tool, which means:
  - If Kafka goes down, application features (like case submission and updates) will be unavailable.
  - There is no SLA for when the service will be restored.

Team members involved in this discussion

Nebula	Turbo	InterOps	QA	Leadership / Stakeholder / SA
Prashant	Chirag	Sharath	Suma	Ashok
Bill	Edgard	Shree	Ajit	Edward
Noah	Umair	Ashwini		Mitch
Kalani	Justina	Manish		Ketan
Kevin	Brett	Vishal Bosale		Sabrina
Jeff				David

Stakeholder Attestation

- [ ] @David Peters
- [ ] @Sabrina Nunez